

Full Stack Development Coding Standards, Real Scenarios & Problem-Solving Guide

Frontend • Backend • Database — Real Production Problems,
Performance Bottlenecks, Root Causes, Solutions & Why They Happen,
30 Full-Stack Scenario Questions & Answers

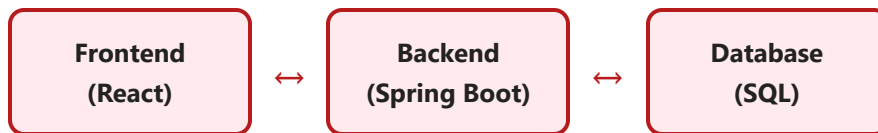
Table of Contents

1. Introduction — Why Standards & Real Scenarios Matter
2. Frontend Coding Standards
3. Backend Coding Standards
4. Database Coding Standards
5. Naming Conventions Across the Stack
6. Code Review Standards & Checklist
7. API Contract Standards Between Frontend & Backend
8. Git & Branching Standards
9. Frontend Real Problems — Performance & Rendering
10. Frontend Real Problems — State, Memory & Data Sync
11. Frontend Real Problems — Build, Deployment & Browser
12. Backend Real Problems — Database Access Layer
13. Backend Real Problems — Concurrency & Threads
14. Backend Real Problems — Memory & Resource Leaks
15. Backend Real Problems — API Design & Integration
16. Database Real Problems — Slow Queries
17. Database Real Problems — Locking & Deadlocks
18. Database Real Problems — Schema & Migrations
19. Database Real Problems — Data Integrity
20. Cross-Cutting Problem — CORS & Frontend-Backend Integration
21. Cross-Cutting Problem — Authentication Across the Stack
22. Cross-Cutting Problem — Caching Consistency
23. Cross-Cutting Problem — Data Consistency Across Layers
24. Performance Bottleneck Methodology — How to Actually Find Them
25. Performance Bottleneck Real Scenario — Frontend Rendering
26. Performance Bottleneck Real Scenario — Network & API Latency
27. Performance Bottleneck Real Scenario — Backend CPU & I/O
28. Performance Bottleneck Real Scenario — Database Query Plans
29. Scalability Challenges & Real Scenarios
30. Security Challenges Across the Full Stack

- 31. Common Full-Stack Anti-Patterns to Avoid
- 32. Real-World Case Study 1 — Flash Sale Checkout Meltdown
- 33. Real-World Case Study 2 — The Slow Dashboard That Got Worse Every Week
- 34. Real-World Case Study 3 — The Silent Data Corruption Bug
- 35. 30 Full-Stack Scenario Questions & Answers
- 36. Quick Revision Summary & Standards Checklist

1. Introduction — Why Standards & Real Scenarios Matter

Individually knowing React, Spring Boot, or SQL isn't the same skill as building and OPERATING a real full-stack application in production. Most real-world engineering problems live at the SEAMS between layers — a frontend assumption about API shape, a backend assumption about data volume, a database assumption about query patterns — and standards exist precisely to reduce how often those seams break.



This guide is deliberately organized around REAL problems and their root causes — not framework tutorials. Every section follows the same pattern: **Issue** (what actually goes wrong), **Why** (the real underlying cause), and **Fix** (the concrete solution) — because understanding WHY something breaks is what lets you prevent the next ten similar bugs, not just patch the one in front of you.

2. Frontend Coding Standards

Standard	Rule	Why It Matters
Component size	Keep components under ~200 lines; extract sub-components when they grow beyond one clear responsibility	Smaller components are easier to test, reuse, and reason about in code review
State location	Keep state as LOCAL as possible; lift it up only when genuinely shared by multiple components	Over-centralizing state causes unrelated components to re-render unnecessarily and makes data flow harder to trace
API calls	Never call fetch/axios directly inside a component body — centralize in a services/api layer	Keeps components focused on rendering; makes API changes a one-place edit, not a hunt across the codebase
Prop typing	Use TypeScript (or PropTypes at minimum) for every component's props	Catches integration mistakes (wrong prop type/missing prop) at compile time instead of a confusing runtime bug
Styling	Pick ONE approach (CSS Modules, Tailwind, styled-components) and use it consistently project-wide	Mixing multiple styling strategies causes specificity conflicts and makes onboarding harder
Accessibility	Every interactive element must be keyboard-reachable; every image needs alt text; every input needs a label	Real legal requirement in many regions, and a real portion of users depend on it
Error handling	Every async operation must handle both loading AND error states explicitly, never just the happy path	Real networks fail; an app that only handles success looks broken the first time an API call is slow or fails

3. Backend Coding Standards

Standard	Rule	Why It Matters
Layered architecture	Strict separation: Controller (HTTP) → Service (business logic) → Repository (data access)	Business logic in controllers becomes untestable without spinning up the full web layer; mixing layers makes changes ripple unpredictably
DTOs, never entities	Never return JPA entities directly from a REST controller	Prevents accidental exposure of internal fields and decouples the API contract from the database schema (changing a column shouldn't break the API)
Transactions	Apply <code>@Transactional</code> at the SERVICE layer, not the controller or repository	The service layer is where a "unit of business work" is naturally defined; transactions at the wrong layer cause partial-commit bugs
Exception handling	One global <code>@RestControllerAdvice</code> handling all exception-to-HTTP-status mapping consistently	Prevents 47 different ad-hoc try/catch blocks each returning a differently-shaped error response
Logging	Structured logging with a correlation/trace ID on every log line, never <code>System.out.println</code>	Essential for debugging a single request's journey across a real multi-service, multi-instance production system
Configuration	Externalize all environment-specific values (URLs, credentials, feature flags) — never hardcode	The exact same JAR must run correctly in dev/QA/prod without code changes, only config changes
Input validation	Validate at the API boundary with Bean Validation (<code>@Valid</code>), never trust client-provided data	The frontend's validation is a UX convenience only — a malicious or buggy client can always bypass it entirely

4. Database Coding Standards

Standard	Rule	Why It Matters
Migrations	Every schema change goes through a versioned migration tool (Flyway/Liquibase) — never manual ad-hoc ALTER statements in production	Guarantees every environment's schema history is identical, reviewable, and reproducible
Indexing	Index columns used in WHERE/JOIN/ORDER BY based on REAL observed query patterns, verified with EXPLAIN	Un-indexed columns cause full table scans that get progressively slower as data grows; over-indexing slows down every write
Foreign keys & constraints	Enforce referential integrity and NOT NULL/CHECK constraints at the database level, not just application code	Application-level-only validation can be bypassed by a bug, a batch job, or a direct DB access tool — the database is the last line of defense
Naming	Consistent snake_case table/column names, singular or plural consistently applied project-wide	Inconsistent naming ("user" vs "Users" vs "user_tbl") causes constant friction and ORM mapping mistakes
Transactions & isolation	Choose the isolation level deliberately per use case, don't just accept the database default blindly	The right isolation level depends on the actual consistency needs of that specific operation (Section 17)
Sensitive data	Encrypt/hash sensitive columns (passwords always hashed, PII encrypted at rest where required)	A database backup or dump leak shouldn't directly expose usable sensitive data

5. Naming Conventions Across the Stack

Layer	Convention	Example
React components/files	PascalCase	<code>ProductCard.jsx</code>
JavaScript variables/functions	camelCase	<code>calculateTotalPrice()</code>
Java classes	PascalCase	<code>OrderService</code>
Java methods/variables	camelCase	<code>findByCustomerId()</code>
REST endpoints	lowercase, plural nouns, kebab-case for multi-word	<code>/api/order-items</code>
Database tables/columns	snake_case	<code>order_items</code> , <code>customer_id</code>
Environment variables	UPPER_SNAKE_CASE	<code>DATABASE_URL</code>

Consistency MATTERS more than which exact convention is chosen — a codebase mixing `camelCase` and `snake_case` arbitrarily forces every developer to constantly context-switch and increases typo-based bugs (a mistyped column name that "looks right").

6. Code Review Standards & Checklist

- Does this change include tests for the new behavior (unit tests for logic, integration tests for API contracts)?
- Are all new database queries checked for N+1 problems and missing indexes?
- Is user input validated on the backend, not just trusted from the frontend?
- Are secrets/credentials absent from the diff (no hardcoded API keys, passwords)?
- Does the change handle the error/failure path, not just the happy path?
- Is the change backward-compatible with existing API consumers, or is a version bump needed?
- Are new dependencies genuinely necessary, and free of known critical vulnerabilities?

A code review checklist exists to catch the SAME categories of mistakes repeatedly made across a team — if your team keeps hitting the same bug class in production, add it to this list explicitly rather than relying on reviewers to remember it informally.

7. API Contract Standards Between Frontend & Backend

Standard	Rule
Consistent response envelope	Every endpoint returns the same top-level shape (e.g., <code>{ data, error, meta }</code>) across the entire API
Consistent error format	Every error response includes a machine-readable code AND a human-readable message, not just a stack trace
Versioning discipline	Breaking changes require a new API version (<code>/api/v2/</code>); additive changes (new optional fields) don't
Documentation as code	OpenAPI/Swagger generated directly from backend code, never a hand-maintained separate document that drifts out of sync
Contract testing	Automated tests (e.g., Pact) verify the frontend's ACTUAL expectations against the backend's real responses in CI

Real-time example: A backend team renamed a field from `userName` to `username` as a "small cleanup" — with no contract test in place, this silently broke the frontend in production, since the frontend was reading the old field name and got `undefined` everywhere it was displayed. A contract test would have failed the backend's own CI build before the change ever merged.

8. Git & Branching Standards

Standard	Rule
Branch naming	<code>feature/JIRA-123-short-description</code> , <code>hotfix/...</code> , <code>bugfix/...</code>
Commit messages	Imperative mood, reference a ticket ID, describe WHY not just WHAT ("Fix N+1 query in order history, JIRA-456")
PR size	Keep pull requests small and focused — a 2000-line PR is nearly impossible to review meaningfully
Required checks	CI (build, tests, lint, security scan) must pass before merge is even allowed, enforced by branch protection

9. Frontend Real Problems — Performance & Rendering

ISSUE A product listing page with 2,000 items becomes sluggish to scroll and interact with.

WHY React re-renders and the browser lays out/paints every single DOM node, even ones far off-screen and invisible to the user — rendering 2,000 real DOM nodes is expensive regardless of how fast the underlying logic is.

FIX Implement virtualization (e.g., `react-window`) so only the visible ~20 rows are actually rendered to the DOM at any time, recycling DOM nodes as the user scrolls.

ISSUE Typing in a search box causes visible input lag on every keystroke.

WHY Every keystroke triggers an immediate API call AND a full re-render of a large results list, competing with the browser's ability to keep up with fast typing.

FIX Debounce the API call (wait until typing pauses) and ensure the results list itself is componentized so unrelated parts of the page don't re-render on every keystroke.

ISSUE A dashboard with several charts takes 4+ seconds to become interactive after navigation.

WHY All chart data is fetched sequentially (one await after another) instead of concurrently, and the entire chart library is bundled into the main JS chunk loaded upfront regardless of whether the user ever visits that page.

FIX Fetch independent data sources concurrently with `Promise.all`, and code-split the charting library behind a dynamic `import()` so it only loads when that specific page is actually visited.

10. Frontend Real Problems — State, Memory & Data Sync

ISSUE After editing an item in a list, the UI shows outdated data until a manual page refresh.

WHY The edit API call succeeded, but the LOCAL state/cache holding the list was never updated or invalidated to reflect the change — the UI is reading from stale client-side state, not the fresh server truth.

FIX After a successful mutation, either update local state directly with the response data or invalidate/refetch the relevant cached query (e.g., React Query's cache invalidation) so the UI reflects the true current server state.

ISSUE A long-running single-page application (open for hours) gradually consumes more and more browser memory.

WHY Event listeners, timers, or subscriptions set up when a component mounted were never cleaned up when it unmounted — each navigation away and back creates MORE leaked references, none of which the garbage collector can ever reclaim.

FIX Always return a cleanup function from `useEffect` that removes listeners/clears timers, matching every subscription with its corresponding teardown.

ISSUE Two different pages show conflicting values for "the same" piece of data (e.g., cart item count).

WHY Multiple independent local component states are each tracking their own COPY of what should be one shared piece of truth, with no single source of truth keeping them synchronized.

FIX Lift genuinely shared state into a single source of truth (Context or a state-management library like Redux Toolkit) instead of letting multiple components independently track duplicated copies.

11. Frontend Real Problems — Build, Deployment & Browser

ISSUE Users report seeing a broken/half-updated app right after a new deployment.

WHY The browser has an old `index.html` cached, referencing JS/CSS bundle filenames that no longer exist on the server after the new deployment replaced them.

FIX Use content-hashed filenames for bundles (so old and new can coexist briefly) and configure short/no-cache headers specifically on `index.html`, so it's always fetched fresh while the hashed assets it references can be cached long-term.

ISSUE The application works perfectly in Chrome but breaks in Safari for some users.

WHY A JavaScript or CSS feature used is not yet supported (or behaves differently) in Safari's engine — a real, common cross-browser compatibility gap, especially with newer language/CSS features.

FIX Check caniuse.com before adopting a new feature, ensure Babel/Autoprefixer are correctly configured for your actual target browser list, and include real cross-browser testing in the QA process, not just Chrome.

12. Backend Real Problems — Database Access Layer

ISSUE An order-history API endpoint that was fast in testing becomes extremely slow in production as data grows.

WHY The classic N+1 query problem — fetching 50 orders triggers 1 query for the orders, then 1 ADDITIONAL query per order to lazily load its items, becoming 51 total queries instead of 1-2, and scaling linearly worse as the result set grows.

FIX Use a JOIN FETCH query or an Entity Graph to retrieve orders and their items in one (or a couple of) queries instead of N+1 — always verify with SQL logging/EXPLAIN during development, not just functional testing.

ISSUE The application suddenly throws "Cannot get a connection, pool error" under moderate load.

WHY Connection pool exhaustion — either the pool size is genuinely too small for real concurrent load, or (far more commonly) connections are being held open too long/leaked due to a missing transaction boundary or a slow query blocking the connection.

FIX Monitor actual connection pool utilization under real load, size the pool based on measured concurrency needs (not a guessed default), and audit for any code path acquiring a connection without reliably releasing it.

ISSUE A batch job updating thousands of rows takes hours and locks up the database for other users.

WHY The entire batch runs as ONE giant transaction, holding locks on every touched row for the full multi-hour duration instead of committing incrementally.

FIX Process the batch in smaller chunks with periodic commits (exactly the chunk-oriented processing model in Spring Batch), releasing locks incrementally instead of holding everything hostage for hours.

13. Backend Real Problems — Concurrency & Threads

ISSUE Two customers both successfully "buy" the last unit of a popular product during a flash sale, resulting in oversold inventory.

WHY A classic race condition — both requests read the SAME stock count (e.g., 1) before either one's update committed, so both see "available" and proceed, unaware of the other's concurrent action.

FIX Use optimistic locking (a version column, retried on conflict) or pessimistic locking (SELECT ... FOR UPDATE) on the inventory row during the check-and-decrement operation, ensuring it's treated as one atomic unit.

ISSUE Under high traffic, the application becomes completely unresponsive — even simple, unrelated requests start timing out.

WHY A slow downstream dependency (a third-party API, a slow query) is holding request-handling threads hostage waiting for a response; once ALL available threads are blocked waiting on the slow dependency, even completely unrelated fast requests have no thread left to be handled by.

FIX Apply strict timeouts and circuit breakers on every outbound call, and use bulkheads to isolate thread pools per dependency, so one slow dependency can't exhaust threads needed for unrelated requests.

14. Backend Real Problems — Memory & Resource Leaks

ISSUE A Spring Boot service's memory usage climbs steadily over days until it eventually crashes with an `OutOfMemoryError`.

WHY Commonly an unbounded in-memory cache/collection that keeps growing with no eviction policy, or a resource (file handle, HTTP client connection) opened but never properly closed in a finally block or try-with-resources.

FIX Profile with a heap dump analysis tool to find the actual growing object type, add a bounded/LRU eviction policy to any cache, and audit resource lifecycle management (always try-with-resources for closeable resources).

ISSUE A container gets OOMKilled by Kubernetes despite the JVM reporting "normal" heap usage internally.

WHY An older JVM doesn't automatically respect the container's cgroup memory limit, instead sizing its heap based on the HOST machine's total memory — leading the JVM to use more memory than the container is actually allowed, triggering an OS-level kill outside the JVM's own awareness.

FIX Use a modern JDK version with container-awareness enabled, and explicitly set `-XX:MaxRAMPercentage` aligned with the pod's configured memory limit.

15. Backend Real Problems — API Design & Integration

ISSUE A "small" backend change breaks a mobile app version that hasn't been updated by users yet.

WHY A field was renamed or removed without considering that OLDER, already-installed client versions are still calling the SAME API and expect the OLD shape — unlike a web frontend, you can't force every user to instantly get the new client version.

FIX Treat API changes as strictly additive by default; use versioning for genuinely breaking changes, and maintain old API versions until client adoption data confirms it's safe to retire them.

ISSUE A payment gets processed twice because the client retried a timed-out request.

WHY The first request actually succeeded server-side, but the RESPONSE was lost in transit (network blip) before reaching the client — the client, seeing no response, retried the exact same logical operation, and the API wasn't designed to recognize it as a duplicate.

FIX Implement an idempotency-key pattern on payment/critical write endpoints — the server stores and recognizes a previously-processed key, returning the original result instead of repeating the side effect.

16. Database Real Problems — Slow Queries

ISSUE A query that used to run in milliseconds now takes several seconds after months in production.

WHY Table growth over time means a previously "fast enough without an index" full table scan now has to scan millions more rows — performance problems from missing indexes are often invisible in development/testing with small data volumes and only surface as real data accumulates.

FIX Run `EXPLAIN / EXPLAIN ANALYZE` on the slow query to confirm a full table scan, then add an index on the actual filtered/joined columns — always load-test with **REALISTIC** data volumes, not just a handful of sample rows.

ISSUE Adding an index didn't speed up a query that filters on that exact column.

WHY The query wraps the indexed column in a function (`WHERE UPPER(email) = ...`) or compares it against a mismatched data type — both prevent the database from actually using the index, since the index was built on the **RAW** column value, not the transformed one.

FIX Rewrite the predicate to avoid wrapping the indexed column in a function (transform the input value instead), or create a function-based/expression index that matches the exact query pattern used.

17. Database Real Problems — Locking & Deadlocks

ISSUE Two concurrent transactions occasionally fail with a "deadlock detected" error.

WHY Transaction A locks Row 1 then waits for Row 2; Transaction B locks Row 2 then waits for Row 1 — each is waiting for a lock the other is holding, with neither able to proceed; the database detects this cycle and force-kills one transaction to break it.

FIX Enforce a CONSISTENT lock acquisition order across all transactions touching the same tables (e.g., always lock accounts in ascending ID order for a transfer operation), eliminating the possibility of a circular wait.

ISSUE Choosing SERIALIZABLE isolation for "maximum safety" causes frequent transaction failures under real concurrent load.

WHY SERIALIZABLE is the strictest isolation level, and under genuine concurrent access to overlapping data, the database must abort and retry transactions to preserve that strict guarantee — a direct, expected trade-off of choosing maximum consistency.

FIX Choose isolation level DELIBERATELY per actual use case rather than defensively defaulting to the strictest option everywhere — most operations are correctly served by READ COMMITTED with targeted optimistic locking (Section 13) only where real conflict risk exists.

18. Database Real Problems — Schema & Migrations

ISSUE A schema migration that worked fine in staging causes a production outage during deployment.

WHY Staging's much smaller table meant an `ALTER TABLE ADD COLUMN` (or worse, a column type change) that locks the table for its full duration completed in milliseconds — the SAME migration against a production table with 200 million rows locks it for MINUTES, during which all reads/writes to that table block.

FIX Test migrations against production-scale data volumes (or a realistic clone) before deploying, and use online schema-change tools/patterns (adding nullable columns, backfilling in batches) for large tables instead of a single blocking ALTER statement.

ISSUE Different environments (dev, QA, prod) have subtly different database schemas, causing "works in QA, fails in prod" bugs.

WHY Manual, ad-hoc schema changes were applied directly to some environments and not others, with no single source of truth tracking exactly which changes have been applied where.

FIX Enforce that ALL schema changes go exclusively through a versioned migration tool (Flyway/Liquibase) applied identically and in the same order to every environment — never a manual console change, ever.

19. Database Real Problems — Data Integrity

ISSUE

Orphaned records are discovered — order line items referencing a deleted product that no longer exists.

WHY

No foreign key constraint enforced the relationship at the database level — a product was deleted directly (perhaps via a batch cleanup script) without the application logic that **WOULD** have prevented/cascaded it correctly being involved at all.

FIX

Add a proper foreign key constraint with an explicit ON DELETE behavior (RESTRICT to prevent deletion while references exist, or CASCADE/SET NULL if that's genuinely the intended business behavior) — never rely on application code alone to enforce referential integrity.

ISSUE

A customer's account balance is occasionally slightly incorrect after concurrent transactions.

WHY

A "read balance, calculate new balance, write new balance" sequence isn't atomic — a second concurrent transaction can read the balance **BETWEEN** the first transaction's read and write, both computing based on the same stale starting value, and the **LAST** write silently overwrites the other's update (a "lost update").

FIX

Perform the update as a single atomic SQL statement (`UPDATE accounts SET balance = balance - 100 WHERE id = ?`) rather than separate read-then-write application code, or use optimistic locking with a version column to detect and retry lost-update conflicts.

20. Cross-Cutting Problem — CORS & Frontend-Backend Integration

ISSUE API calls work perfectly from Postman but fail with a CORS error from the actual React app.

WHY CORS is enforced ONLY by browsers on JavaScript-initiated requests — Postman is not a browser and never applies this check, so a CORS misconfiguration is invisible there and only surfaces when a real browser makes the same call.

FIX Configure the backend's CORS policy to explicitly allow the frontend's exact production and development origins — this can only ever be fixed on the SERVER side, never the client.

21. Cross-Cutting Problem — Authentication Across the Stack

ISSUE Users are randomly logged out under moderate traffic, even though their session should still be valid.

WHY Session state is stored in each backend instance's local memory, but a load balancer routes the user's NEXT request to a DIFFERENT instance that has never heard of that session.

FIX Move to stateless JWT-based authentication (no server-side session to lose), or externalize session storage to a shared store like Redis accessible by every instance.

22. Cross-Cutting Problem — Caching Consistency

ISSUE A product's price was updated in the database, but the storefront keeps showing the OLD price for several minutes.

WHY A cache (Redis, browser cache, or CDN edge cache) is still serving a previously-cached response, and nothing explicitly invalidated/evicted that specific cached entry when the underlying data changed.

FIX Explicitly evict/update the relevant cache entry as part of the SAME write operation that changes the price (cache-aside with write-through invalidation), rather than relying purely on a TTL to eventually expire it.

23. Cross-Cutting Problem — Data Consistency Across Layers

ISSUE An order shows as "Paid" in the frontend, but the Inventory Service never actually reserved the stock.

WHY The checkout flow spans multiple services/steps (Order → Payment → Inventory) with no distributed transaction coordinating them — a failure partway through (Inventory Service briefly down) leaves the system in a genuinely inconsistent state across services.

FIX Implement the Saga pattern with explicit compensating transactions for each step, so a failure anywhere in the sequence triggers a defined rollback of the steps that already succeeded, rather than leaving permanent inconsistency.

24. Performance Bottleneck Methodology — How to Actually Find Them



Tool	Reveals
Browser DevTools (Network/Performance tabs)	Frontend rendering time, individual API call latency, largest resources slowing page load
Distributed tracing (Jaeger/Zipkin/App Insights)	Exactly WHICH backend service/hop is slow in a multi-service request
Database EXPLAIN/EXPLAIN ANALYZE	Whether a specific query is using indexes correctly or doing a full table scan
APM tools (New Relic, Dynatrace)	End-to-end request breakdown across frontend, backend, and database in one unified view

The single most important rule of performance work: Never optimize based on assumption — a developer's intuition about "what's probably slow" is wrong surprisingly often. Always MEASURE first, fix the genuinely confirmed bottleneck, then measure AGAIN to confirm the fix actually helped before moving to the next suspected issue.

25. Performance Bottleneck Real Scenario — Frontend Rendering

ISSUE A React dashboard feels sluggish; DevTools Performance recording shows most time spent in "Scripting", not network or painting.

WHY React Profiler reveals a top-level component re-renders on every state change anywhere in the app, and because it's not memoized, EVERY child component re-renders too, even ones whose actual props never changed.

FIX Wrap expensive child components in `React.memo`, move state as low/local as possible so unrelated parts of the tree don't re-render, and memoize callback props with `useCallback` so memoization actually takes effect.

26. Performance Bottleneck Real Scenario — Network & API Latency

ISSUE A page makes 15 separate small API calls sequentially, each waiting for the previous one, taking 3+ seconds total even though each call individually is fast.

WHY Round-trip network LATENCY (not the server's processing time) dominates when many small calls are made one after another — even a 100ms round trip, repeated 15 times sequentially, adds up to 1.5 seconds of pure waiting.

FIX Fire independent requests CONCURRENTLY with `Promise.all` instead of sequentially, or better, have the backend expose a single aggregated endpoint (or a BFF/GraphQL layer) returning everything the page needs in one round trip.

27. Performance Bottleneck Real Scenario — Backend CPU & I/O

ISSUE A report-generation endpoint pegs CPU at 100% and slows down every other request on that instance.

WHY A synchronous, CPU-heavy computation (generating a large PDF/Excel report) runs directly on a request-handling thread, competing for CPU with every other concurrent request on the same instance.

FIX Move heavy report generation to an asynchronous background job (a queue-backed worker), returning a "processing" response immediately and notifying/polling for completion, so it no longer competes with real-time request handling.

28. Performance Bottleneck Real Scenario — Database Query Plans

ISSUE A search feature that filters by multiple optional fields becomes slow for certain filter combinations but fast for others.

WHY A single index only helps the query patterns matching its leftmost-column order — filter combinations not aligned with the existing index fall back to a full or partial table scan, while others happen to benefit from it.

FIX Analyze the ACTUAL most common real filter combinations (from real usage logs, not guesses) and design composite indexes specifically matching them, or use a dedicated search engine (Elasticsearch) for genuinely flexible multi-field search at scale.

29. Scalability Challenges & Real Scenarios

ISSUE A service that worked fine for months suddenly can't handle a marketing-driven traffic spike.

WHY The service holds session/local state in memory, preventing horizontal scaling (adding more instances doesn't help if a user's data only exists on ONE specific instance) — vertical scaling (a bigger single server) eventually hits a hard ceiling.

FIX Make the service genuinely stateless (externalize any session/cache to Redis) so it can scale horizontally behind a load balancer, combined with autoscaling rules tested BEFORE the real traffic spike, not during it.

30. Security Challenges Across the Full Stack

Layer	Common Real Vulnerability	Fix
Frontend	Rendering unsanitized user content via innerHTML (XSS)	Use <code>textContent</code> for plain text; sanitize any HTML via a trusted library before rendering
Backend	Trusting client-side validation as the only check	Always re-validate every input server-side (Bean Validation), regardless of frontend checks
Database	SQL injection via string-concatenated queries	Always use parameterized queries/JPA — never concatenate raw user input into SQL
Cross-cutting	Secrets committed to source control or hardcoded in frontend env variables	Use a secrets manager (Key Vault); remember frontend env vars are PUBLICLY visible in the shipped bundle

31. Common Full-Stack Anti-Patterns to Avoid

- **"It works on my machine"** — no containerization/consistent environment, config drift between dev and production.
- **God components/God services** — a single React component or Spring Boot service trying to do everything, impossible to test or reason about.
- **Shared mutable database across services** — multiple "independent" microservices secretly coupled through one shared database table.
- **No monitoring until an incident** — discovering a performance problem only when a customer complains, instead of via proactive dashboards/alerts.
- **Premature optimization without measurement** — spending a week optimizing a function that was never actually the bottleneck.
- **Ignoring the error path** — an app that only demos well on the happy path and falls apart the first time a real network hiccup or invalid input occurs.

32. Real-World Case Study 1 — Flash Sale Checkout Meltdown

Scenario: An e-commerce site's flash sale caused checkout to fail for ~30% of users, with some customers being charged without receiving an order confirmation.

Layer	Root Cause Found
Database	Inventory decrement wasn't atomic — a classic lost-update race condition under high concurrency (Section 19)
Backend	No idempotency key on the payment endpoint — network retries from slow mobile connections caused duplicate charges (Section 15)
Frontend	The "Place Order" button wasn't disabled after the first click, letting frustrated/impatient users click multiple times during the slowdown

FIX Atomic inventory updates with optimistic locking, an idempotency-key pattern on the payment endpoint, AND a disabled submit button after first click — a real production incident rarely has just ONE root cause; this one needed fixes at all three layers simultaneously.

33. Real-World Case Study 2 — The Slow Dashboard That Got Worse Every Week

Scenario: An internal analytics dashboard started fast, but load time steadily crept up week over week until it was unusable (12+ seconds).

Layer	Root Cause Found
Database	A reporting query joined across the LIVE transactional tables with no index supporting the specific date-range filter used — as the table grew, the full scan got linearly worse
Backend	The same heavy query ran directly against production for every dashboard page load, with zero caching of results between users viewing the same date range

FIX Applied CQRS — moved reporting to a separate, denormalized read model updated via events, added the missing composite index, and cached common date-range queries for a short TTL — load time dropped from 12 seconds to under 200ms.

34. Real-World Case Study 3 — The Silent Data Corruption Bug

Scenario: Months after launch, an audit discovered a small percentage of customer orders had an incorrect total, with no errors ever logged anywhere.

Layer	Root Cause Found
Frontend	The order total displayed to the user was calculated CLIENT-SIDE and sent to the backend, which trusted it directly instead of recalculating
Backend	A discount-stacking bug in the frontend's calculation (allowing two discounts to combine when only one should apply) silently produced a slightly wrong total that the backend never double-checked

FIX

The backend now ALWAYS recalculates the order total independently from trusted server-side data (product prices, valid discount rules) — the frontend's calculation is treated purely as a preview/UX convenience, never as the authoritative source, closing an entire category of "trusting the client" bugs at once.

35. 30 Full-Stack Scenario Questions & Answers

Q1. A React page suddenly shows "Cannot read properties of undefined" right after a backend field was renamed. What's the real fix, beyond patching this one crash?

A: Beyond the immediate frontend fix, the real fix is process: introduce contract testing (Section 7) between frontend and backend so a backend field rename is caught in CI before it ever reaches production, not discovered by users.

Q2. Users complain a page is slow. How do you determine whether it's a frontend, network, or backend problem before touching any code?

A: Open the browser Network tab first — check the actual API call durations (backend/network) versus how long the browser spends rendering after the response arrives (frontend); this single check immediately narrows down which layer to investigate further.

Q3. A checkout flow occasionally double-charges customers. What's the layered fix across frontend and backend?

A: Frontend: disable the submit button immediately after the first click. Backend: implement an idempotency-key pattern on the payment endpoint so even a genuine retry from a flaky network never processes the same logical payment twice.

Q4. A list page is fast with 50 items in QA but unusable with 50,000 items in production. What's the real problem and the fix?

A: QA tested with unrealistic data volume — the real fix is backend pagination (never return all rows at once) combined with frontend list virtualization for whatever page size is actually rendered, and a standing practice of testing with production-scale data.

Q5. Two microservices both need to update related data as part of one logical business operation, but each has its own database. How do you keep them consistent?

A: Implement the Saga pattern — a sequence of local transactions per service, each with a defined compensating action, so a failure partway through triggers a deliberate rollback of the already-completed steps instead of leaving permanent cross-service inconsistency.

Q6. A deploy goes out and CPU usage on the backend instances spikes immediately. How do you diagnose it without guessing?

A: Use APM/profiling tools to get an actual CPU flame graph from the affected instances, identifying the specific method consuming the most CPU time — comparing it against the deploy's changed code narrows down the regression precisely, rather than reverting blindly.

Q7. A form's "Save" button appears to do nothing when clicked, with no visible error anywhere. Walk through your debugging approach.

A: Check the browser Console for a JavaScript error first, then the Network tab to see if the API call was even

sent and what it returned — a silent failure usually means either an uncaught exception in the click handler, or an error response that the frontend code isn't actually surfacing to the user.

Q8. Why would a query that's fast for one customer be slow for another, using the exact same code?

A: Data SKEW — one customer might have 10 orders while another has 500,000; an unindexed or poorly-indexed query scales with that specific customer's actual data volume, so "works for most users, slow for a few power users" is a classic sign of a missing index on a filtered column.

Q9. How would you prevent a single slow third-party payment gateway from taking down your entire checkout service?

A: Wrap the outbound call in a timeout, retry (only if safe/idempotent), and circuit breaker, with a bulkhead isolating its thread pool from unrelated request handling — so a slow/down gateway degrades gracefully instead of exhausting all available threads.

Q10. A customer reports their account balance is wrong after making two transactions "at the same time" on two different devices. What's the likely root cause?

A: A lost-update race condition — both devices' requests read the same starting balance before either write committed, and the last write silently overwrote the first; fixed with an atomic UPDATE statement or optimistic locking with a version column.

Q11. Why might a page work fine for you locally but fail with a CORS error for your teammate?

A: Likely each of you is running the frontend on a different local port, or one is testing against a different backend environment whose CORS configuration doesn't include that specific origin — CORS errors are entirely about the exact origin making the request.

Q12. How would you design an API and database schema to support both a web app and a mobile app safely evolving independently?

A: Version the API explicitly for breaking changes, keep changes additive by default (new optional fields, not renamed/removed ones), and never assume all clients are on the latest version — maintain old API versions until real usage data confirms it's safe to retire them.

Q13. A batch job that updates millions of rows overnight sometimes fails halfway through, leaving data partially updated. How do you make it safely resumable?

A: Process in smaller committed chunks (Spring Batch's chunk-oriented model) with tracked progress, so a failure can resume from the last successfully committed chunk instead of either losing all progress or needing a single giant all-or-nothing transaction.

Q14. Why does a feature that seems to work in every manual test still fail intermittently in production?

A: Manual testing rarely exercises genuine CONCURRENCY — race conditions, connection pool exhaustion

under real parallel load, or timing-dependent bugs often only surface under real simultaneous multi-user traffic, which single-person manual testing can't replicate.

Q15. How would you investigate a memory leak reported in a long-running Spring Boot service?

A: Take heap dumps before and after a period of suspected leaking activity, compare object counts to find what's growing unexpectedly, then trace those objects back to the code path retaining references to them (commonly an unbounded cache or an unclosed resource).

Q16. A React app's bundle size keeps growing every sprint, slowing down initial load. How do you address it systematically, not just once?

A: Add bundle size analysis to the CI pipeline with a size budget that fails the build if exceeded, and use code-splitting for rarely-visited routes/heavy libraries — making bundle size a continuously enforced standard, not a one-time cleanup.

Q17. Why should a frontend never trust its own calculated total/price when submitting an order?

A: Frontend code and network requests can be modified by a user via browser DevTools or tools like Postman — the backend must always independently recalculate critical values from trusted server-side data, treating the frontend's value as a preview only.

Q18. A new index was added to speed up a slow query, but write performance on that table got noticeably worse. Why, and is this expected?

A: Yes, expected — every index must also be updated on every INSERT/UPDATE/DELETE affecting that table, so indexes trade write performance for read performance; add indexes deliberately based on real query patterns, not defensively on every column.

Q19. How would you debug why a specific user can't log in, when login works for everyone else?

A: Check backend logs filtered by that user's ID/email for the exact failure reason (account locked, expired token, wrong hash comparison), and check the Network tab for the exact response the frontend actually received — user-specific bugs are almost always data-specific, not code-specific.

Q20. Why might enabling HTTPS actually break a previously-working feature, and how do you fix it?

A: "Mixed content" — a page loaded over HTTPS trying to load a resource (image, script, API call) over plain HTTP is blocked by the browser by default; ensure every resource/API reference uses HTTPS consistently once the site itself is served over HTTPS.

Q21. A support team reports "the app is slow" with no other detail. What questions do you ask before touching any code?

A: Which specific page/action, since when, for all users or specific ones, and do they have a timestamp/screenshot — vague reports need to be narrowed to a specific, reproducible scenario before any meaningful investigation (Section 24's methodology) can begin.

Q22. How do you prevent a single misbehaving API consumer from degrading service for everyone else?

A: Enforce rate limiting at the API Gateway, scoped per API key/client, so one partner's excessive or buggy traffic pattern can't exhaust shared resources needed by every other consumer.

Q23. Why would the exact same SQL query have two completely different execution plans on two different days?

A: The database's query planner uses table statistics (row counts, data distribution) that can become stale after significant data changes — running `ANALYZE` /updating statistics after major data changes helps the planner choose the genuinely optimal plan again.

Q24. A team wants to migrate a monolith's single shared database into separate databases per microservice. What's the biggest real risk during this migration?

A: Data synchronization gaps during the transition period, where both the old and new systems might accept writes — mitigated with Change Data Capture (CDC) for near-real-time replication and a clearly-defined single-writer cutover boundary, not a "big bang" switch.

Q25. How would you design the frontend to handle an API that occasionally takes 10+ seconds to respond, without the user thinking the app is frozen?

A: Show an explicit loading state immediately (not just a blank screen), set a reasonable client-side timeout with a friendly retry option, and consider an async pattern (202 Accepted + polling) on the backend for genuinely long operations rather than holding the HTTP connection open.

Q26. Why might two developers on the same team introduce the same category of bug repeatedly, and how do you actually fix this at the team level, not just the code level?

A: A recurring bug CLASS (not just one bug) signals a missing standard or review checklist item — add it explicitly to the code review checklist (Section 6) and consider automated linting/static analysis to catch it before human review even happens.

Q27. A production incident is traced to a developer accidentally running a destructive script against the production database directly. What process change prevents this specific category of incident?

A: No developer should have direct write access to production databases for routine changes — enforce that ALL schema/data changes go through reviewed, versioned migrations (Section 18) and a controlled deployment pipeline, removing the possibility of an ad-hoc direct script entirely.

Q28. How do you decide whether a performance problem should be fixed by scaling infrastructure (more servers) versus optimizing code?

A: If the bottleneck is a genuinely inefficient algorithm/query (e.g., an $N+1$ problem or a missing index), more servers won't help and will just mask the underlying inefficiency at higher cost; scale infrastructure only after confirming the code/query path itself is already reasonably efficient for the actual workload.

Q29. A React Context used for global app state causes many unrelated components to re-render whenever ANY part of it changes. How would you fix this without abandoning Context entirely?

A: Split the one large Context into several smaller, more focused Contexts (e.g., separate Auth and Theme contexts) so a component only subscribes to and re-renders from the specific slice it actually needs, or migrate genuinely large/frequently-changing state to Redux Toolkit's more granular useSelector subscriptions.

Q30. Summarize, in one sentence each, the most important standard for each layer of the stack covered in this guide.

A: Frontend — always handle loading and error states, never just the happy path. Backend — never trust client input, always validate and recalculate critical logic server-side. Database — enforce every schema change through versioned migrations and verify query plans before shipping to production.

36. Quick Revision Summary & Standards Checklist

- Standards exist to prevent the SAME bug class recurring across a team — when a bug repeats, add it to the checklist, don't just fix the instance.
- Most real production problems live at the SEAMS between frontend, backend, and database — not inside any single layer in isolation.
- Never trust the frontend for critical calculations/validation — the backend must always be the final source of truth.
- N+1 queries, missing indexes, and connection pool exhaustion are the most common real backend/database performance killers.
- Race conditions (lost updates, overselling) require atomic operations or explicit locking — "it usually works" is not a fix.
- Always MEASURE before optimizing — use Network tabs, distributed tracing, and EXPLAIN plans, never guesswork.
- CORS errors are fixed only on the backend; frontend "protected routes" are UX only, never real security.
- Test with production-scale data volumes — most "fast in QA, slow in production" bugs are really "small data vs big data" bugs in disguise.
- Every schema change goes through versioned migrations — never a manual, ad-hoc production database edit.
- A real production incident usually has root causes at MULTIPLE layers simultaneously — fix all of them, not just the most visible one.